สะพานย้อนวิถี (reverse_bridge)



เมื่อคำสาปคลาย สะพานสายโยงก็พลิกกลับทิศ ตราที่เคยอยู่ท้ายสายกลายเป็นตราแรก และทุกอย่างย้อนกลับ อรุณต้องกลับสายเชื่อมของตราทั้งหมด (หรือเพียงบางช่วง) เพื่อให้ขบวนข้ามสะพานได้ การกลับสายต้องทำโดยเปลี่ยนเส้นทางของลูกศรเท่านั้น หาก "สลับรหัสบนตรา" หรือ "หลอมตราขึ้นใหม่" ตราดั้งเดิมจะสูญเสียพลังและสะพานจะพังลง

สิ่งที่ฝึกในด่านนี้: กลับลำดับลิสต์: `prev` / cur / nxt, กลับเฉพาะช่วงด้วย dummy node

รายละเอียดการเขียนโปรแกรม

เขียนฟังก์ชันที่ประกาศในไฟล์ `reverse_bridge.h` ต่อไปนี้ โดยไม่ต้องเขียน `main` และไม่ต้องอ่านหรือพิมพ์ข้อมูลทางอินพุตและเอาต์พุตมาตรฐาน (โปรแกรมตรวจของผู้จัดทำหน้าที่นั้น)

```
struct Node { int data; Node* next; };
void reverseList(Node*& head);
void reverseRange(Node*& head, int l, int r);
```

- `reverseList(head)`: กลับลำดับของโหนดทั้งลิสต์ โดยเปลี่ยนเฉพาะ `next` และ `head`
- `reverseRange(head, l, r)`: กลับลำดับเฉพาะโหนดตำแหน่งที่ l ถึง r (นับจาก 1) โดยโหนดอื่นอยู่ที่เดิม รับประกัน $1 \le l \le r \le$ จำนวนโหนด
- ห้ามแก้ `data` ห้ามสร้างโหนดใหม่ และห้ามเรียก `new` ทั้งโดยตรงและโดยอ้อม (เช่น ใช้ `vector` หรือ `string`) ภายในสองฟังก์ชันนี้ — โปรแกรมตรวจนับการจองหน่วยความจำ
- หลังทุกคำสั่ง โปรแกรมตรวจเทียบ "ตัวโหนด" ทุกตัว (ไม่ใช่แค่ค่า) กับลำดับที่ควรเป็น และไม่อนุญาตให้โหนดใดหายหรือถูก `delete`
- ต้องทำได้ในเวลา `O(N)` ต่อคำสั่ง และใช้หน่วยความจำเพิ่ม `O(1)`

ก่อน: 1 → 2 → 3 → 4 → 5



หลัง `reverseList(head)`: ได้ 5 → 4 → 3 → 2 → 1 (โหนดเดิมทั้งห้า เปลี่ยนเฉพาะลูกศรและ `head`)



หลัง `reverseRange(head, 2, 4)` จากลิสต์ก่อนหน้า: ได้ $1 \to 4 \to 3 \to 2 \to 5$



รูปที่ 1 ผลของการกลับทั้งลิสต์และกลับเฉพาะช่วง (โหนดสีเขียวคือโหนดในช่วงที่ถูกกลับ)

การส่งงาน: ดาวน์โหลดไฟล์แนบ `reverse_bridge.cpp` (ไฟล์โครง) และ `reverse_bridge.h` จากหน้าโจทย์ แล้วเขียนฟังก์ชันลงในไฟล์ `reverse_bridge.cpp` ให้เสร็จ จากนั้นส่งเฉพาะไฟล์ `reverse_bridge.cpp` ห้ามแก้ไขไฟล์ `reverse_bridge.h` ระบบจะคอมไพล์ไฟล์ของคุณพร้อมกับ `grader.cpp` ของผู้จัด ไฟล์ `attachment.zip` มีโปรแกรมตรวจสำหรับทดลองในเครื่อง (`g++ -std=c++17 -O2 grader.cpp reverse_bridge.cpp -o reverse_bridge`) ซึ่งตรวจการใช้ `new` / `delete` และความถูกต้องของโหนดให้เหมือนกับระบบจริง

ตัวอย่างการเรียกฟังก์ชัน

การเรียกฟังก์ชัน (ลิสต์ก่อนเรียก)	ผลที่ได้
`reverseList(head)` เมื่อลิสต์คือ $1 \to 2 \to 3$	ได้ $3 \to 2 \to 1$ (`head` ชี้โหนดที่เคยเป็นตัวท้าย)
`reverseList(head)` เมื่อลิสต์คือ 9	ได้ 9 (ลิสต์โหนดเดียว)
`reverseList(head)` เมื่อ `head` เป็น `nullptr`	ลิสต์ว่างเหมือนเดิม
`reverseRange(head, 2, 4)` เมื่อลิสต์คือ $1 \to 2 \to 3 \to 4 \to 5$	ได้ $1 \to 4 \to 3 \to 2 \to 5$
`reverseRange(head, 1, 3)` เมื่อลิสต์คือ $1 \to 2 \to 3 \to 4$	ได้ $3 \to 2 \to 1 \to 4$ (`head` เปลี่ยน)
`reverseRange(head, 3, 3)` เมื่อลิสต์คือ $1 \to 2 \to 3 \to 4$	ไม่เปลี่ยน (ช่วงยาว 1 โหนด)

ตารางแสดงตัวอย่างการเรียกที่แยกจากกัน

ข้อจำกัด

- $1 \le Q \le 100$
- $0 \le N \le 200\ 000$ และ $0 \le M \le 50$ ในแต่ละชุด
- ผลรวมของ N ทุกชุดในกรณีทดสอบเดียวกันไม่เกิน 300 000
- $-10^9 \le \texttt{data} \le 10^9$ และทุกคำสั่ง `R l r` มี $1 \le l \le r \le N$
- เวลา 2 วินาที และหน่วยความจำ 256 MiB ต่อกรณีทดสอบ

ปัญหาย่อย

1. (20 คะแนน) มีเฉพาะคำสั่ง `W`, $N \le 10$ และ $M \le 5$
2. (30 คะแนน) มีเฉพาะคำสั่ง `W` (N ใหญ่ได้ถึง 200 000)
3. (50 คะแนน) ไม่มีข้อจำกัดเพิ่มเติม (มีคำสั่ง `R` ได้)

แต่ละปัญหาย่อยคิดคะแนนแยกกัน จะได้รับคะแนนของปัญหาย่อยเมื่อผ่านกรณีทดสอบทุกกรณีในปัญหาย่อยนั้น

รูปแบบข้อมูลสำหรับโปรแกรมตรวจตัวอย่าง

บรรทัดแรก: จำนวนเต็ม Q คือจำนวนชุดข้อมูล
แต่ละชุด: บรรทัดแรกมี N และ M บรรทัดถัดไปมีค่าของลิสต์ N ค่า (บรรทัดว่างถ้า $N = 0$) แล้วตามด้วยคำสั่ง M บรรทัด: `W` เรียก `reverseList(head)` · `R l r` เรียก `reverseRange(head, l, r)`
โปรแกรมตรวจทำคำสั่งตามลำดับ (ตรวจโครงสร้างหลังทุกคำสั่ง) แล้วพิมพ์ 1 บรรทัดต่อหนึ่งชุด: N ตามด้วยค่า `data` ของทุกโหนดจากหัวถึงท้ายหลังจบคำสั่งทั้งหมด

ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

ข้อมูลนำเข้า 1	ผลลัพธ์ 1
2 5 2 1 2 3 4 5 R 2 4 W 0 1 W	5 5 2 3 4 1 0

ชุดแรก: กลับช่วงตำแหน่ง 2–4 ได้ 1 4 3 2 5 แล้วกลับทั้งลิสต์ได้ 5 2 3 4 1 · ชุดที่สองเรียก `reverseList` กับลิสต์ว่าง ต้องไม่เกิดข้อผิดพลาด
ตัวอย่างนี้สอดคล้องกับปัญหาย่อยที่ 3

ข้อมูลนำเข้า 2	ผลลัพธ์ 2
2 5 1 3 1 4 1 5 W 1 2 8 W W	5 5 1 4 1 3 1 8

มีเฉพาะคำสั่ง `W` ค่า 1 ซ้ำกันไม่มีผลกับการกลับ เพราะโปรแกรมตรวจเทียบตัวโหนด
ตัวอย่างนี้สอดคล้องกับปัญหาย่อยที่ 1, 2, 3